

2. Проектирование взаимодействия

2.1 Протоколы и паттерны взаимодействия в целевой архитектуре AdScale

2.1.1 Внешний протокол интеграции с DSP

Протокол: HTTP/2 + JSON (OpenRTB 2.5+)

Обоснование:

- OpenRTB является индустриальным стандартом для Real-Time Bidding, поддерживается всеми крупными DSP и SSP.
- Использование JSON упрощает отладку и интеграцию с партнёрами, не требует генерации клиентского кода.
- HTTP/2 позволяет мультиплексировать запросы и уменьшить накладные расходы.
- При необходимости для продвинутых партнёров можно предоставить gRPC + Protobuf как альтернативный эндпоинт, но базовым остаётся OpenRTB.

2.1.2 Внутренние протоколы взаимодействия между микросервисами

Сервис ставок (критический путь, $\leq 80\text{--}100$ мс P95) → gRPC

Протокол: gRPC поверх HTTP/2, сообщения в Protocol Buffers.

- Request/Response (синхронно) для самого аукциона.
- Circuit Breaker на вызовах зависимостей (чтобы не «положить» аукцион, если Redis или PostgreSQL временно тормозит).
- Cache-aside (сначала Redis, потом PostgreSQL, затем обновление кэша).
- Retry с экспоненциальной задержкой для кратковременных сбоев.

Обоснование:

- Низкая задержка: Бинарная сериализация Protobuf значительно быстрее JSON, а HTTP/2 позволяет использовать постоянные соединения и мультиплексирование;
- Строгие контракты: .proto-файлы обеспечивают типизацию и генерацию кода на многих языках, упрощая поддержку;
- Управление тайм-аутами и отменами: gRPC поддерживает контекстные тайм-ауты и отмену запросов, критично для соблюдения SLA;

2.1.3. Асинхронное взаимодействие (событийно-ориентированное)

Протокол: Apache Kafka

Обоснование:

- Разделение критического пути: Отправка событий в Kafka производится асинхронно и не блокирует ответ аукциона.
- Буферизация и сглаживание пиков: Kafka выступает как буфер, позволяя потребителям (статистика, биллинг) обрабатывать данные в своём темпе.
- Гарантии доставки: Поддержка at-least-once / exactly-once семантики, возможность replay при сбоях.
- Масштабируемость: Независимые группы потребителей позволяют горизонтально масштабировать каждый сервис-потребитель
- Устойчивость к отказам: Если потребитель временно недоступен, сообщения не теряются и могут быть обработаны позже.

Формат: Avro + Schema Registry — чтобы контролировать эволюцию схем и избежать «сломанных» сообщений при изменениях контрактов.

2.2 API Gateway

API Gateway обязателен в целевой архитектуре

Обоснование:

- Единая точка входа для всех внешних запросов (DSP, рекламодатели, внутренние системы).
- Маршрутизация: Направляет запросы к соответствующим микросервисам (/rtb/bid → Bidding Ingress, /api/campaigns → Campaign Rules и т.д.).
- Безопасность: TLS-терминация, аутентификация (JWT/API Keys), rate limiting, защита от DDoS.
- Управление трафиком: Canary-релизы, A/B-тестирование, откаты через динамическую маршрутизацию.
- Наблюдаемость: Сбор метрик, логирование, распределённая трассировка (через интеграцию с OpenTelemetry).
- В перспективе - Geo-DNS: Направление запросов в ближайший регион для снижения задержек.

2.3 Итог

Взаимодействие	Протокол	Обоснование
Внешнее (DSP → AdScale)	OpenRTB (HTTP/2 + JSON)	Индустриальный стандарт, совместимость, простота.

Внутреннее синхронное (RTB-путь)	gRPC	Минимальная задержка, строгие контракты, поддержка потоков и таймаутов.
Внутреннее синхронное (UI, администрирование)	REST (HTTP/JSON)	Простота, удобство для фронтенда, OpenAPI-документация.
Внутреннее асинхронное (события)	Kafka	Буферизация, надежность, replay, разделение критического пути.